

The Complete Trees Roadmap (Beginner → Advanced, Java)

97 curated problems, verified LeetCode numbers, organized by conceptual dependency. 🍀 = Premium. All Java templates use:

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode() {}
    TreeNode(int val) { this.val = val; }
    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val; this.left = left; this.right = right;
    }
}
```

Level 0 — Prerequisites (no problems — confirm you have these first)

- Recursion (base case, recursive case, call stack)
 - Stack & Queue (push/pop, offer/poll)
 - HashMap / HashSet
 - Arrays and basic loops
 - Basic DFS/BFS on graphs (helps, not required)
 - References/pointers (Java passes object references — this matters a lot for tree mutation)
 - Big-O basics
-
-

Master Roadmap by Level

Level 1 — Tree Fundamentals

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
1	104	Maximum Depth of Binary Tree	Easy	Basic DFS / Depth	Level 0	1
2	965	Univalued Binary Tree	Easy	Basic Recursion / Property Check	#1	2
3	111	Minimum Depth of Binary Tree	Easy	Basic DFS / Depth (BFS also valid)	#1	3
4	404	Sum of Left Leaves	Easy	Leaf Identification	#1	4
5	222	Count Complete Tree Nodes	Medium	Node Counting (+ optimization trick)	#1	5
6	559	Maximum Depth of N-ary Tree	Easy	N-ary Tree Basics	#1	6

Level 2 — Tree Traversals (the three DFS orders)

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
7	144	Binary Tree Preorder Traversal	Easy	DFS / Preorder	Level 1	7
8	94	Binary Tree Inorder Traversal	Easy	DFS / Inorder	Level 1	8
9	145	Binary Tree Postorder Traversal	Easy	DFS / Postorder	Level 1	9
10	589	N-ary Tree Preorder Traversal	Easy	DFS / N-ary	#7	10
11	590	N-ary Tree Postorder Traversal	Easy	DFS / N-ary	#9	11

(Solve 144/94/145 recursively first, then again iteratively with an explicit `Stack<TreeNode>` — see Section "Recursive vs Iterative" below.)

Level 3 — BFS / Level Order

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
12	102	Binary Tree Level Order Traversal	Medium	BFS / Queue	Level 2	12

13	107	Binary Tree Level Order Traversal II	Medium	BFS + Reverse	#12	13
14	103	Binary Tree Zigzag Level Order Traversal	Medium	BFS + Direction Flip	#12	14
15	637	Average of Levels in Binary Tree	Easy	BFS + Aggregation	#12	15
16	515	Find Largest Value in Each Tree Row	Medium	BFS + Aggregation	#12	16
17	429	N-ary Tree Level Order Traversal	Medium	BFS / N-ary	#12	17
18	1161	Maximum Level Sum of a Binary Tree	Medium	BFS + Aggregation	#12	18

Level 4 — Basic DFS Patterns (combine left + right results)

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
19	100	Same Tree	Easy	Recursive DFS / Compare	Level 2	19
20	101	Symmetric Tree	Easy	Recursive DFS / Mirror Compare	#19	20
21	226	Invert Binary Tree	Easy	Recursive DFS / Mutate	#19	21
22	110	Balanced Binary Tree	Easy	Recursive DFS + Early Combine	#1	22
23	572	Subtree of Another Tree	Easy	Recursive DFS + Same-Tree reuse	#19	23
24	617	Merge Two Binary Trees	Easy	Recursive DFS / Combine	#19	24
25	872	Leaf-Similar Trees	Easy	DFS + Collect-and-Compare	Level 2	25

Level 5 — Root-to-Leaf / Path Problems

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
---	------	--------------	------------	---------	--------------	-------

26	112	Path Sum	Easy	DFS + Running Sum (Root→Leaf)	Level 4	26
27	257	Binary Tree Paths	Easy		#26	27
28	113	Path Sum II	Medium	DFS + Backtracking (collect paths)	#26–27	28
29	129	Sum Root to Leaf Numbers	Medium	DFS + Running Value	#26	29
30	1022	Sum of Root To Leaf Binary Numbers	Medium	DFS + Running Value (bit form)	#29	30
31	437	Path Sum III	Medium	DFS + Prefix Sum + HashMap (any→any node!)	#26, Prefix Sum	31
32	988	Smallest String Starting From Leaf	Medium	DFS + Backtracking (leaf→root string)	#27	32
33	666	Path Sum IV	Medium	DFS + Encoded Tree	#26	33
34	1457	Pseudo- Palindromic Paths in a Binary Tree	Medium	DFS + Bitmask (root→leaf)	#26, bitmask	34

Level 6 — Binary Search Tree

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
35	700	Search in a Binary Search Tree	Easy	BST Search	Level 4	35
36	701	Insert into a Binary Search Tree	Medium	BST Insert	#35	36
37	98	Validate Binary Search Tree	Medium	BST Validation (bounds-passing DFS)	#35	37
38	108	Convert Sorted Array to Binary Search Tree	Easy	Build BST from sorted data	#35	38
39	109	Convert Sorted List to Binary Search Tree	Medium	Build BST from sorted data	#38	39
40	230	Kth Smallest Element in a BST	Medium	Inorder = sorted order	#37	40

41	530	Minimum Absolute Difference in BST	Easy	Inorder = sorted order	#40	41
42	783	Minimum Distance Between BST Nodes	Easy	Inorder = sorted order (dup. of #41)	#41	42
43	653	Two Sum IV - Input is a BST	Easy	Inorder + Two Pointers / HashSet	#40	43
44	938	Range Sum of BST	Easy	BST-pruned DFS	#35	44
45	235	Lowest Common Ancestor of a Binary Search Tree	Easy	BST property (compare vs root)	#35	45
46	450	Delete Node in a BST	Medium	BST Delete (successor swap)	#36	46
47	669	Trim a Binary Search Tree	Medium	BST + Recursive Restructure	#46	47
48	173	Binary Search Tree Iterator	Medium	Inorder + Controlled Iteration (Stack)	#40	48

49	897	Increasing Order Search Tree	Easy	Inorder + Restructure	#40	49
50	1038	Binary Search Tree to Greater Sum Tree	Medium	Reverse Inorder + Running Sum	#40	50
51	99	Recover Binary Search Tree	Hard	Inorder + Swap Detection	#40	51
52	96	Unique Binary Search Trees	Medium	Tree-shaped DP (Catalan number)	DP basics	52
53	95	Unique Binary Search Trees II	Medium	Tree Construction + DP	#52	53
54	1008	Construct Binary Search Tree from Preorder Traversal	Medium	BST Construction	#35–36	54
55	1382	Balance a Binary Search Tree	Medium	Inorder + Rebuild (like #38)	#38, #40	55

56	1305	All Elements in Two Binary Search Trees	Medium	Inorder + Merge	#40	56
57	449	Serialize and Deserialize BST	Medium	BST-specific Serialization	#37, Level 9	57
58	285 9	Inorder Successor in BST	Medium	BST property navigation	#40	58

Level 7 — Tree Views & Structural Problems

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
59	199	Binary Tree Right Side View	Medium	BFS (last-per-level) or DFS (right-first)	Level 3	59
60	662	Maximum Width of Binary Tree	Medium	BFS + Position Indexing	Level 3	60
61	314 9	Binary Tree Vertical Order Traversal	Medium	BFS + HashMap (column index)	Level 3, HashMap	61

62	987	Vertical Order Traversal of a Binary Tree	Hard	DFS/BFS + Coordinates + Sorting	#61	62
63	545	Boundary of Binary Tree	Medium	DFS (left bound + leaves + right bound)	Level 4	63

(Left Side View isn't a separate LeetCode problem — solve it as a mirrored variation of #199 for practice.)

Level 8 — Lowest Common Ancestor

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
64	236	Lowest Common Ancestor of a Binary Tree	Medium	LCA / Recursive Return-Up 🔥 CORE	Level 4	64
—	235	<i>(cross-ref, Level 6)</i>	Easy	LCA in BST (simpler — use ordering)	—	—
65	1650	Lowest Common Ancestor of a Binary Tree III	Medium	LCA with Parent Pointers (two-pointer)	#64, LinkedList intersection idea	65

66	1123	Lowest Common Ancestor of Deepest Leaves	Medium	LCA + Depth Tracking (return tuple)	#64	66
67	865	Smallest Subtree with all the Deepest Nodes	Medium	Same problem as #1123, different phrasing	#66	67
68	863	All Nodes Distance K in Binary Tree	Medium	LCA-adjacent: build parent map + BFS	#64, BFS	68

Level 9 — Tree Construction

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
69	105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Construction + HashMap indices 🔥 CORE	Level 2	69

70	106	Construct Binary Tree from Inorder and Postorder Traversal	Medium	Construction (mirror of #69)	#69	70
71	889	Construct Binary Tree from Preorder and Postorder Traversal	Medium	Construction (no unique solution guarantee)	#69	71
—	1008	(<i>cross-ref, Level 6</i>)	Medium	BST-specific construction (simpler)	—	—
—	108	(<i>cross-ref, Level 6</i>)	Easy	Construction from sorted array	—	—
72	297	Serialize and Deserialize Binary Tree	Hard	Full Tree Serialization 🔥 CORE	#69	72
—	449	(<i>cross-ref, Level 6</i>)	Medium	BST-specific serialization (no null markers needed)	—	—
73	606	Construct String from Binary Tree	Easy	Reverse Operation (tree → string)	Level 2	73

74	1028	Recover a Tree From Preorder Traversal	Hard	Construction from Depth-Encoded String	#69	74
----	------	--	------	--	-----	----

Level 10 — Tree Transformation

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
—	226	<i>(cross-ref, Level 4)</i>	Easy	Invert (in-place mutation)	—	—
75	114	Flatten Binary Tree to Linked List	Medium	DFS + In-place Restructure	Level 4	75
—	617	<i>(cross-ref, Level 4)</i>	Easy	Merge (combine two trees)	—	—
—	669	<i>(cross-ref, Level 6)</i>	Medium	Trim (BST-specific)	—	—
—	450	<i>(cross-ref, Level 6)</i>	Medium	Delete (BST-specific)	—	—
76	1325	Delete Leaves With a Given Value	Medium	Postorder + Conditional Delete	Level 4	76

77	1110	Delete Nodes And Return Forest	Medium	Postorder + Forest Building	#76	77
78	814	Binary Tree Pruning	Medium	Postorder + Conditional Delete	#76	78
79	623	Add One Row to Tree	Medium	BFS/DFS + Structural Insert	Level 4	79
80	116	Populating Next Right Pointers in Each Node	Medium	BFS or O(1)-space level-linking	Level 3	80
81	117	Populating Next Right Pointers in Each Node II	Medium	Same idea, imperfect tree (harder)	#80	81

Level 11 — Advanced Tree DP

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
82	543	Diameter of Binary Tree	Easy	Tree DP: global answer + return-up value 🔥 CORE	Level 4	82

83	687	Longest Univalued Path	Medium	Tree DP (diameter variant, with condition)	#82	83
84	124	Binary Tree Maximum Path Sum	Hard	Tree DP: global max + "best downward branch" 🔥 CORE	#82	84
85	337	House Robber III	Medium	Tree DP: choose/don't-choose, return pair	#82, DP basics	85
86	979	Distribute Coins in Binary Tree	Medium	Tree DP: postorder flow/balance accounting	#82	86
87	968	Binary Tree Cameras	Hard	Tree DP: 3-state greedy (postorder)	#86	87
88	1339	Maximum Product of Splitted Binary Tree	Medium	Tree DP: total sum + subtree-sum search	#82, total-sum precompute	88
89	1372	Longest ZigZag Path in a Binary Tree	Medium	Tree DP: direction-aware state	#82	89

90	1026	Maximum Difference Between Node and Ancestor	Medium	Tree DP: pass down min/max so far	#82	90
----	------	--	--------	-----------------------------------	-----	----

Level 12 — Advanced / Hard Trees

#	LC #	Problem Name	Difficulty	Pattern	Prerequisite	Order
91	958	Check Completeness of a Binary Tree	Medium	BFS + null-marker completeness check	Level 3	91
92	951	Flip Equivalent Binary Trees	Medium	DFS + Either-Orientation Compare	Level 4	92
93	652	Find Duplicate Subtrees	Medium	Postorder + Serialization + HashMap	Level 9, Level 4	93
94	366 9	Find Leaves of Binary Tree	Medium	DFS + Height-Bucketed Collection	Level 4	94
95	1361	Validate Binary Tree Nodes	Medium	Tree-as-Graph validation (in-degree + cycle)	Graph basics	95
96	1080	Insufficient Nodes in Root to Leaf Paths	Medium	Postorder + Path-Sum Pruning	Level 5, Level 10	96
—	124	(<i>cross-ref, Level 11</i>)	Hard	Capstone Tree DP	—	—

—	297	(cross-ref, Level 9)	Hard	Capstone Construction	—	—
—	987	(cross-ref, Level 7)	Hard	Capstone Views	—	—
—	968	(cross-ref, Level 11)	Hard	Capstone Tree DP	—	—

Recommended Solving Order — The Curriculum

- 1–6 (Level 1).** Start with **104 (Max Depth)** — write `1 + max(dfs(left), dfs(right))` and *feel* what a recursive return value is. **965, 111, 404, 222, 559** are reps of the same "combine children" shape with small twists (a property check, a min instead of max, a leaf condition, an optimization, an N-ary child list instead of two fixed children).
- 7–11 (Level 2).** **144/94/145** — memorize the three orders by writing them from scratch, not by pattern-matching the mnemonic. Do each recursively, then immediately redo iteratively with a `Stack<TreeNode>` (see Section 11). **589/590** confirm you understand traversal generalizes to N children.
- 12–18 (Level 3).** **102 (Level Order)** is the archetype: `Queue<TreeNode>, process size = queue.size()` nodes per iteration to know where a level ends. Everything else in Level 3 (**107, 103, 637, 515, 429, 1161**) reuses this exact skeleton with a different thing computed per level.
- 19–25 (Level 4).** This is where "what does DFS(node) return?" starts to matter. **100 (Same Tree)** returns a boolean built from `left match && right match && val match`. **101 (Symmetric)** is #100 but comparing a tree against its own mirror. **226 (Invert)** teaches that a recursive call can *mutate* structure, not just read it. **110 (Balanced), 572 (Subtree), 617 (Merge), 872 (Leaf-Similar)** are variations — by #25 you

should be able to sketch the recursive shape before writing code.

26–34 (Level 5). 112 (Path Sum) is your first "carry state down the recursion" problem: subtract `node.val` from the target as you descend, check `target == node.val` at leaves. **257, 113, 129, 1022** vary what's carried (a string path, a list of paths, a running number). **437 (Path Sum III) is the pivotal problem here** — it's *not* root-to-leaf, so the naive DFS breaks, and you need Prefix Sum + HashMap (same trick as LC 560 if you've done the Prefix Sum roadmap) to count any-node-to-any-node paths in $O(n)$. This is the clearest place to feel the difference between the two path types the prompt asked about.

35–58 (Level 6, the biggest level — budget real time here). Start with **700/701 (Search/Insert)** to internalize `val < node.val → go left`. **98 (Validate BST)** is the first BST problem that trips people up — you must pass down a (`lower`, `upper`) bound, not just compare `node.val` to its immediate children. **108/109** build a balanced BST from sorted data (bisect the middle). **230 (Kth Smallest)** is where you prove to yourself that **inorder traversal of a BST visits nodes in sorted order** — this single fact powers **530, 783, 653, 173, 897, 1038, 99, 1382, 1305**. **235 (LCA in BST)** shows the BST-specific shortcut (compare values, no need to search both subtrees) versus the general **236** you'll hit in Level 8. **450/669 (Delete/Trim)** are the hardest *mechanics* in this level — deleting a BST node requires finding the inorder successor. **96/95 (Unique BST counts)** quietly previews Tree DP. Close the level with **1008, 449, 285** for construction/serialization/navigation specific to BSTs.

59–63 (Level 7). 199 (Right Side View) — solve it two ways: BFS (take the last node of each level) and DFS (visit right child before left, record first node seen at each depth). Both are correct; knowing both is the point. **662, 314, 987, 545** each add one more piece of bookkeeping (position index, column index + sort, or boundary-specific traversal order).

64–68 (Level 8). 236 (LCA of Binary Tree) is a must-master template: recurse into both children; if both return non-null, the current node *is* the LCA; otherwise propagate whichever side found something. **1650** swaps this for a two-pointer/parent-pointer technique (same idea as detecting a linked-list intersection). **1123/865** teach you to return a

(node, depth) pair instead of a single value — your first taste of "return more than one piece of information," which is essential for Level 11. **863** builds a parent map via DFS, then BFS's outward from the target node.

69–74 (Level 9). 105 (Preorder+Inorder → Tree) is the archetype: preorder's first element is the root; find that value's index in inorder (HashMap for $O(1)$ lookup) to split left/right subtrees. **106, 889** are direct variations from the other traversal pairs. **297 (Serialize/Deserialize) is a must-master capstone** — preorder with explicit null markers, split on a delimiter, rebuild recursively consuming a shared index/queue. **606, 1028** round out construction from the opposite direction.

75–81 (Level 10). 114 (Flatten) — do it with $O(1)$ extra space using the "Morris-style" right-child rewiring, not just an ArrayList dump, to really learn in-place mutation. **1325, 1110, 814** are the same shape: postorder DFS, decide at each node whether to keep/detach it based on children's results. **116/117** teach connecting nodes at the same level without a queue ($O(1)$ space) by using already-established `.next` pointers from the level above.

82–90 (Level 11 — the heart of "hard" tree interviews). 543 (Diameter) is the single most important template in this entire roadmap. Learn the shape cold:

```
int diameter = 0;
int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    int right = height(node.right);
    diameter = Math.max(diameter, left + right); // update
    return 1 + Math.max(left, right);           // return
}
```

Every problem after this reuses "global variable + return-to-parent" with a different combine rule: **687** adds a value-match condition, **124** swaps "height" for "best downward path sum" (and clips negative branches to 0), **337** returns a {rob, don't-rob} pair instead of a single number, **979/968** flip to postorder "balance/state passed up," and **1339/1372/1026** each carry a different piece of state down or up.

91–96 (Level 12 — capstones). 958, 951, 652, 366, 1361, 1080 each combine two earlier skills (BFS+indexing, DFS+either-orientation compare, postorder+serialization+hashmap, DFS+bucketing, graph validation, postorder+pruning). By this point you should be able to look at each and name the two prerequisite patterns it's built from before you start coding.

Must-Do vs ☆ **Recommended** vs **Optional** vs **Hard Challenge**

 **Must-Do (26)** 104, 226, 100, 101, 110, 144, 94, 145, 102, 112, 257, 700, 98, 230, 235, 236, 105, 297, 543, 124, 337, 199, 108, 236 (*again — core enough to bear repeating*), 450, 968

☆ **Recommended (37)** 965, 111, 404, 222, 559, 589, 590, 107, 103, 637, 515, 572, 617, 872, 113, 129, 437, 701, 108, 109, 530, 653, 938, 669, 173, 897, 1038, 96, 106, 114, 1325, 814, 116, 687, 979, 1339, 1372

 **Optional (24)** 429, 1161, 1022, 988, 666, 1457, 783, 95, 1008, 1382, 1305, 449, 285, 314, 545, 662, 1123, 865, 863, 889, 606, 1028, 623, 117

 **Hard Challenge (13)** 99, 1650, 987, 958, 951, 652, 366, 1361, 1080, 1026, 968, 124, 297

(A few of these — 968, 124, 297 — also double as Level 11/9 Must-Do capstones; they're listed here too because they're where the difficulty spikes hardest.)

Tree Pattern Cheat Sheet

Basic DFS

- Recognize: "visit every node", compute something from subtree results.
- Core idea: recurse into children, combine.
- Time: $O(n)$. Space: $O(h)$ call stack (h = height; $O(n)$ worst case for a skewed tree).

```
int dfs(TreeNode node) {
    if (node == null) return 0;           // base case
    int left = dfs(node.left);
    int right = dfs(node.right);
    return 1 + Math.max(left, right);    // combine
}
```

Preorder (Root → Left → Right)

- Use when: you need to process a node *before* its children (e.g., copying/serializing top-down, or you need the root's value before descending).

```
void preorder(TreeNode n, List<Integer> out) {
    if (n == null) return;
    out.add(n.val);
    preorder(n.left, out);
    preorder(n.right, out);
}
```

Inorder (Left → Root → Right)

- Use when: the tree is a BST and you need sorted order.

```
void inorder(TreeNode n, List<Integer> out) {
    if (n == null) return;
    inorder(n.left, out);
    out.add(n.val);
    inorder(n.right, out);
}
```

Postorder (Left → Right → Root)

- Use when: a node's decision depends on its *children's* results first (deletion, Tree DP, height/diameter, bottom-up aggregation).

```
void postorder(TreeNode n, List<Integer> out) {
    if (n == null) return;
    postorder(n.left, out);
    postorder(n.right, out);
    out.add(n.val);
}
```

BFS / Level Order

- Recognize: "level by level", "row by row", "zigzag", "top-to-bottom width".
- Time: $O(n)$. Space: $O(n)$ (queue can hold up to a full level, up to $n/2$ nodes).

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;
    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode n = q.poll();
            level.add(n.val);
            if (n.left != null) q.offer(n.left);
            if (n.right != null) q.offer(n.right);
        }
        res.add(level);
    }
    return res;
}
```

Root-to-Leaf

- Recognize: "root to leaf", "path from root", sum/string built along

one fixed direction.

- Core idea: carry state (running sum, path list/string) as a parameter down the recursion; check condition only at leaves (`node.left == null && node.right == null`).

```
void dfs(TreeNode n, int remaining, List<Integer> path, List<Integer> res) {
    if (n == null) return;
    path.add(n.val);
    remaining -= n.val;
    if (n.left == null && n.right == null && remaining == 0)
        res.add(new ArrayList<>(path));    // copy! path is shared
    } else {
        dfs(n.left, remaining, path, res);
        dfs(n.right, remaining, path, res);
    }
    path.remove(path.size() - 1);          // backtrack
}
```

Path Sum (any-node-to-any-node — Prefix Sum on Trees)

- Recognize: "any node to any node" (not root-to-leaf), counting paths with a target sum.
- Core idea: same `prefix[j] - prefix[i] = target` trick as array prefix sums, but the "array" is the current root-to-node path; use a `HashMap` of `{running_sum: frequency}`, add on entry, remove on backtrack out.

```
int count(TreeNode node, long sum, long target, Map<Long, Integer> seen) {
    if (node == null) return 0;
    sum += node.val;
    int res = seen.getOrDefault(sum - target, 0);
    seen.merge(sum, 1, Integer::sum);
    res += count(node.left, sum, target, seen) + count(node.right, sum, target, seen);
    seen.merge(sum, -1, Integer::sum);    // backtrack: remove
    return res;
}
```

LCA (Lowest Common Ancestor)

- Recognize: "common ancestor", "closest ancestor of two nodes".
- Core idea (general binary tree): recurse both sides; if both return non-null, current node is the answer; else propagate the non-null side.

```
TreeNode lca(TreeNode node, TreeNode p, TreeNode q) {
    if (node == null || node == p || node == q) return node;
    TreeNode left = lca(node.left, p, q);
    TreeNode right = lca(node.right, p, q);
    if (left != null && right != null) return node;
    return left != null ? left : right;
}
```

- BST shortcut: no need to search both sides — compare `p.val/q.val` against `node.val` and go the direction that keeps both in range, or stop when they diverge.

BST

- Recognize: "binary search tree", "sorted", or a validation/search/insert/delete ask.
- Core idea: `left < root < right` at every node — not just immediate children (this is the #1 bug source in "Validate BST").

```
boolean isValidBST(TreeNode n, long lower, long upper) {
    if (n == null) return true;
    if (n.val <= lower || n.val >= upper) return false;
    return isValidBST(n.left, lower, n.val) && isValidBST(n.right, n.val, upper);
}
```

- **Why inorder of a BST is sorted:** inorder visits Left → Root → Right, and by the BST invariant everything in Left is smaller and everything in Right is larger than Root — so visiting in that order necessarily visits values in ascending order.

Tree Construction

- Recognize: "construct tree from traversals", given two of {preorder, inorder, postorder}.

- Core idea: one traversal tells you the root; the other (usually inorder) tells you how to split into left/right subtrees. HashMap the inorder value → index for $O(1)$ split lookup instead of scanning ($O(n) \rightarrow$ avoids $O(n^2)$).

```
Map<Integer, Integer> idx = new HashMap<>();
int preIdx = 0;
TreeNode build(int[] preorder, int inStart, int inEnd) {
    if (inStart > inEnd) return null;
    int rootVal = preorder[preIdx++];
    TreeNode root = new TreeNode(rootVal);
    int mid = idx.get(rootVal);
    root.left = build(preorder, inStart, mid - 1);
    root.right = build(preorder, mid + 1, inEnd);
    return root;
}
```

Tree Views

- Recognize: "right/left side view", "top/bottom view", "vertical order".
- Core idea: BFS gives you level info for free (side views = first/last node per level); vertical order needs a (column, row) coordinate per node, tracked via DFS or BFS, then grouped/sorted by column.
- Data structure: Queue for BFS-based views; HashMap<Integer, List<int[]>> (column → [row, val]) for vertical order.

Tree DP

- Recognize: "diameter", "maximum path sum", "longest path", "rob/don't-rob a tree", any problem needing a *global* answer computed from *local* subtree results.
- Core idea: postorder DFS; compute a value for each subtree; update a global answer using both children's values; return only what the *parent* actually needs (often a subset or transformed version of what you used for the global answer).

```

int best = Integer.MIN_VALUE;
int dfs(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(dfs(node.left), 0);    // clip neg
    int right = Math.max(dfs(node.right), 0);
    best = Math.max(best, node.val + left + right);    //
    return node.val + Math.max(left, right);    //
}

```

Maximum Path Sum (LC 124 specifically)

- Same template as Tree DP above — the two-line difference from Diameter (LC 543) is: track `node.val + left + right` (values, not heights) for the global answer, and clip negative branches to 0 before combining, since including a negative subtree can only hurt.

Serialization

- Recognize: "serialize", "deserialize", "encode/decode a tree to a string".
- Core idea: preorder traversal with explicit null markers (e.g., "#"), joined by a delimiter; deserialize by consuming tokens in the same preorder sequence.
- Time: $O(n)$ both ways. Space: $O(n)$.

```

String serialize(TreeNode root) {
    if (root == null) return "#";
    return root.val + "," + serialize(root.left) + "," +
}
int i = 0;
TreeNode deserialize(String[] tokens) {
    String tok = tokens[i++];
    if (tok.equals("#")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(tok));
    node.left = deserialize(tokens);
    node.right = deserialize(tokens);
    return node;
}

```

How to Identify the Tree Pattern in an Interview

"Visit every node" → **DFS / BFS** (either works if order doesn't matter; pick DFS for simplicity, BFS if you need level info)

"Level by level" / "row by row" → **BFS + Queue**

"Root to leaf" → **DFS + Backtracking**, carrying state downward, checking only at leaves

"Any node to any node" (not anchored at root or leaf) → **DFS + Prefix Sum + HashMap** (LC 437-style), because a plain root-to-leaf DFS won't cover paths that start/end mid-tree

"Sorted order" in the context of a BST → **Inorder Traversal**

"Closest/common ancestor" → **LCA** (general template if Binary Tree, value-comparison shortcut if BST)

"Longest path" / "maximum path sum" / "diameter" → **Tree DP** (global answer + return-to-parent value)

"Construct from traversals" → **Tree Construction** (HashMap the inorder indices)

"Serialize / encode / decode" → **Preorder + null markers**

"Right/left/top/bottom/vertical view" → **BFS for side views, coordinates (+ HashMap/sort) for vertical/top/bottom**

Distinguishing lookalikes:

Question	Answer
----------	--------

DFS vs BFS?	Need level-by-level info, shortest-path-like behavior, or "first/last in a row" → BFS. Need to combine subtree results, or space is a concern on a wide-but-shallow tree → DFS.
Binary Tree vs BST?	Does the problem state or imply an ordering invariant (<code>left < root < right</code>)? If yes, use BST shortcuts (skip searching both sides, exploit inorder = sorted). If no, you must consider both children generally.
Root-to-leaf vs any-node-to-any-node?	Does the path have to start at the root and/or end at a leaf? If either anchor is relaxed, you likely need Prefix Sum + HashMap, not a plain path-sum DFS.
Recursive DFS vs Iterative DFS?	Recursive is almost always cleaner and preferred unless the interviewer explicitly asks for iterative, or recursion depth risks a stack overflow on a very unbalanced tree (then use an explicit <code>Stack<TreeNode></code>).
Normal DFS vs Tree DP?	Do you need just one value from the whole tree (height, sum, boolean), or do you need a <i>global optimum considering combinations across children</i> (diameter, max path sum)? The latter is Tree DP — you'll need a variable that lives outside the recursive function.
Tree problem vs Graph problem?	A tree is just a connected, acyclic graph with a designated root and no cycles/multiple parents. If the problem gives you parent pointers, an undirected node list, or explicitly allows cycles/multiple parents to check for, treat it as a graph problem (BFS/DFS with a <code>visited</code> set) even if it "looks" tree-shaped — see LC 1361 for exactly this.

Beginner Tree-Solving Strategy

1. **Draw the tree.** On paper or in your head — trees are visual; don't skip this.
2. **Identify the root.** Where does traversal/recursion start?
3. **Identify what the question is actually asking** — a value, a boolean, a modified tree, a list of paths?
4. **Decide DFS or BFS** based on the clues in the table above.
5. **Decide whether recursion is appropriate** (almost always yes for trees).
6. **Define what the recursive function should return.** This is the single most important design decision — write it down in a comment before coding: `// dfs(node) returns: ...`
7. **Identify the base case** — what happens at `null`? This is almost always your first line of code.
8. **Solve the left subtree** (make the recursive call).
9. **Solve the right subtree** (make the recursive call).
10. **Combine the results** according to what the problem needs.
11. **Check whether a global answer is required** (separate from what's returned to the parent) — if yes, use an instance variable or a mutable wrapper (`int[]` of size 1), since Java can't return two things from a primitive-returning method without a helper class.
12. **Implement the solution.**
13. **Analyze time and space complexity** — say the recursion depth (call stack) out loud as part of space complexity; it's easy to forget.
14. **Dry-run on a small tree** (3–5 nodes) by hand before trusting the code.
15. **Re-solve without looking at the solution**, a day or two later.

The most important Tree skill is not memorizing traversal code. It is understanding what information a recursive call should return to its parent. Once you can answer "what does `dfs(node)` return, and what does its parent do with that value?" for any new problem, you've essentially solved it — the code is just typing.

The Recursion Framework

```
int dfs(TreeNode node) {
```

```

    if (node == null) {
        return ...;           // base case – what's the
    }
    int left = dfs(node.left); // trust the recursion:
    int right = dfs(node.right);
    return ...;               // combine left, right,
}

```

Ask yourself, in this order, for every new problem:

1. **What does `dfs (node)` return?** (a height? a boolean? a sum? a pair?)
2. **What does the parent need from its child** to do its own job?
3. **What should be calculated locally** at this node (using `node.val, left, right`)?
4. **Do I need a global variable** because the final answer isn't the same as what gets returned to the parent?

How the template specializes:

Problem type	Base case returns	Combine step	Global variable?
Height	0	<code>1 + max(left, right)</code>	No
Diameter (543)	0	<code>update best = max(best, left+right); return 1 + max(left, right)</code>	Yes
Max Path Sum (124)	0	<code>update best = max(best, val+left+right); return val + max(left, right, 0)</code>	Yes

Balanced Tree (110)	0	if <code>abs(left-right) > 1</code> or either child unbalanced, signal <code>-1</code> ; else <code>1+max(left, right)</code>	No (encode failure in return value)
Path Sum (112)	<code>remaining == 0</code> at a null-child leaf check	pass <code>remaining - node.val</code> down	No
Subtree (572)	n/a (calls a helper <code>isSame</code>)	compare <code>isSame(node, subRoot)</code> OR recurse into children	No
Tree DP (House Robber III, 337)	<code>{0, 0}</code> (rob, don't-rob)	<code>{val + leftSkip + rightSkip, max(left) + max(right)}</code>	No — return value <i>is</i> the answer at the root

Brute Force → Optimized Progression

LC 236 — Lowest Common Ancestor of a Binary Tree

Brute Force: for each node, collect the full root-to-node paths
compare the two paths to find the deepest common ancestor

↓

Notice: you don't need the whole paths — just where they diverge

↓

Recursive DFS returning up: at each node, ask "is p or q in my left subtree?" If both sides report yes, I'm the LCA

LC 105 — Construct Binary Tree from Preorder and Inorder

Brute Force: for each root value (from preorder), linearly scan inorder to find its index and split

↓

Notice: the inorder array doesn't change – you're re-scanning

↓

Precompute a `HashMap<value, index>` once

LC 124 — Binary Tree Maximum Path Sum

Brute Force: for every pair of nodes, find if a path exists and sum it

↓

Notice: any path has a single "highest" node; that node's value is (its value + the best downward path from its left child) + (the best downward path from its right child)

↓

Postorder Tree DP: compute "best downward path sum" per subtree, update a global max at each node in the same pass

LC 297 — Serialize/Deserialize Binary Tree

Brute Force instinct: "store the array representation" – assume a complete tree shape and waste space / lose space on sparse trees

↓

Notice: preorder traversal, if it includes explicit null values, uniquely encodes any tree shape (not just complete ones)

↓

Preorder + "#" markers: serialize is a straightforward process, deserialize replays the same order, consuming tokens

Why this all works, in one sentence: **a tree's structure is defined entirely by parent-child relationships, so any algorithm that processes a node only after (or only before) both its children have been fully handled can compute global properties in a single linear pass, instead of the polynomial-time re-scanning a brute force falls into.**

Recursive vs Iterative DFS vs BFS

Recursive DFS

```
void dfs(TreeNode node) {
    if (node == null) return;
    // process node (preorder position)
    dfs(node.left);
    // process node (inorder position)
    dfs(node.right);
    // process node (postorder position)
}
```

- Time: $O(n)$. Space: $O(h)$ call stack (h = tree height; $O(\log n)$ balanced, $O(n)$ worst case skewed).
- Prefer when: default choice — cleanest code, matches how most people reason about trees. Avoid only if the tree could be extremely unbalanced/deep (risk of `StackOverflowError` in Java, which has a relatively small default stack) or if the interviewer specifically asks for iterative.

Iterative DFS (explicit Stack)

```
List<Integer> preorderIterative(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;
    Deque<TreeNode> stack = new ArrayDeque<>();
    stack.push(root);
    while (!stack.isEmpty()) {
        TreeNode n = stack.pop();
        res.add(n.val);
        if (n.right != null) stack.push(n.right); // push right first
        if (n.left != null) stack.push(n.left);  // so left is popped first
    }
    return res;
}
```

- Time: $O(n)$. Space: $O(h)$ (or $O(n)$ worst case, same as recursion)

— you're just managing the stack yourself instead of relying on the call stack).

- Prefer when: recursion depth is a real risk, or you need fine-grained control mid-traversal (e.g., BST Iterator LC 173, where you pause and resume traversal across multiple method calls — recursion can't pause like that, but a manually-managed stack can).

BFS (Queue)

```
void bfs(TreeNode root) {  
    if (root == null) return;  
    Queue<TreeNode> queue = new LinkedList<>();  
    queue.offer(root);  
    while (!queue.isEmpty()) {  
        TreeNode n = queue.poll();  
        // process n  
        if (n.left != null) queue.offer(n.left);  
        if (n.right != null) queue.offer(n.right);  
    }  
}
```

- Time: $O(n)$. Space: $O(n)$ worst case (a wide tree can have up to $\sim n/2$ nodes on the last level, all sitting in the queue at once).
- Prefer when: the problem is inherently about levels/rows (level order, zigzag, right side view, shortest structural distance), or you want to avoid deep recursion on an unbalanced tree.

Revision Strategy

Stage	When	What to do
First attempt	Day 0	Solve fresh using the 15-step beginner strategy. Write down what <code>dfs(node)</code> returns before coding.

Review 1	Day 1	Re-read your solution; explain out loud what the recursive function returns and why. No re-coding unless shaky.
Review 2	Day 3	Re-solve from scratch, no notes. If stuck >10 min, peek only at "what should this function return," not the code.
Review 3	Day 7	Pattern-recognition only: read the problem, state the pattern (DFS/BFS/Tree DP/LCA/etc.) and the return-value design, without coding.
Review 4	Day 14	Re-solve all Must-Do problems from scratch, full Java code, timed.
Final review	Day 30	Mixed set: 5 random Must-Do + 3 random Hard Challenge, back-to-back, no hints, interview-style pressure.

Classification for revision:

- **Re-solve completely, every cycle:** the 26 🔥 Must-Do problems.
- **Review approach only after Day 3:** most ☆ Recommended problems once solved correctly twice.
- **Pattern-recognition revision only:** 🕒 Optional problems — recognize the pattern if it reappears; don't re-derive from scratch.
- **Optional revision (only for top-tier prep):** 🏆 Hard Challenge problems — revisit only if targeting Google/Meta/Bloomberg-caliber interviews.

Final Summary

Total curated problems: 97 unique (plus internal cross-references for problems that legitimately belong to two levels)

Difficulty	Count (approx.)
Easy	32

Medium	52
Hard	13

Level / Pattern	# of Problems
Level 1 — Fundamentals	6
Level 2 — Traversals	5
Level 3 — BFS/Level Order	7
Level 4 — Basic DFS Patterns	7
Level 5 — Root-to-Leaf/Path	9
Level 6 — BST	24
Level 7 — Tree Views	5
Level 8 — LCA	5 (+1 cross-ref)
Level 9 — Construction	6 (+3 cross-ref)
Level 10 — Transformation	7 (+4 cross-ref)
Level 11 — Tree DP	9
Level 12 — Advanced/Hard	6 (+4 cross-ref)

Bucket	Count
 Must-Do	26
☆ Recommended	38
 Optional	24
 Hard Challenge	13

Time-Based Roadmaps

If you have 7 days — the most important Tree problems: 104, 226, 100, 101, 144, 94, 145, 102, 112, 700, 98, 230, 236, 543, 124

If you have 14 days — essential interview problems (*the 7-day set,*

plus:) 110, 257, 113, 235, 450, 105, 297, 337, 199, 968, 1123, 863

If you have 30 days — strong preparation (*the 14-day set, plus:)* 111, 404, 222, 572, 617, 872, 129, 437, 701, 108, 109, 530, 653, 938, 669, 173, 96, 106, 114, 1325, 814, 116, 687, 979, 1339, 1372

If you have 60 days — complete beginner → advanced roadmap:

The entire 97-problem Master Roadmap above, solved level-by-level in the listed order, including all 🧩 Optional and 🏆 Hard Challenge problems.

Accuracy note: every problem number and title above was checked against LeetCode's own problem pages/descriptions at the time of writing. Premium-only problems are flagged 🍷 (285, 314, 366) — treat those as read-the-approach-and-understand if you don't subscribe. A few duplicate/near-duplicate LeetCode problems are called out explicitly (783≈530, 865≈1123) so you don't waste time solving the same technique twice thinking it's new.